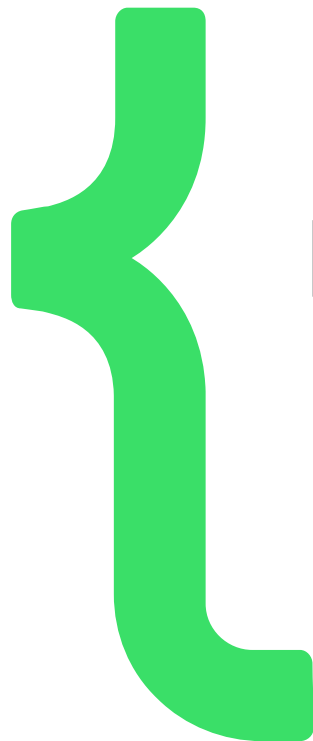




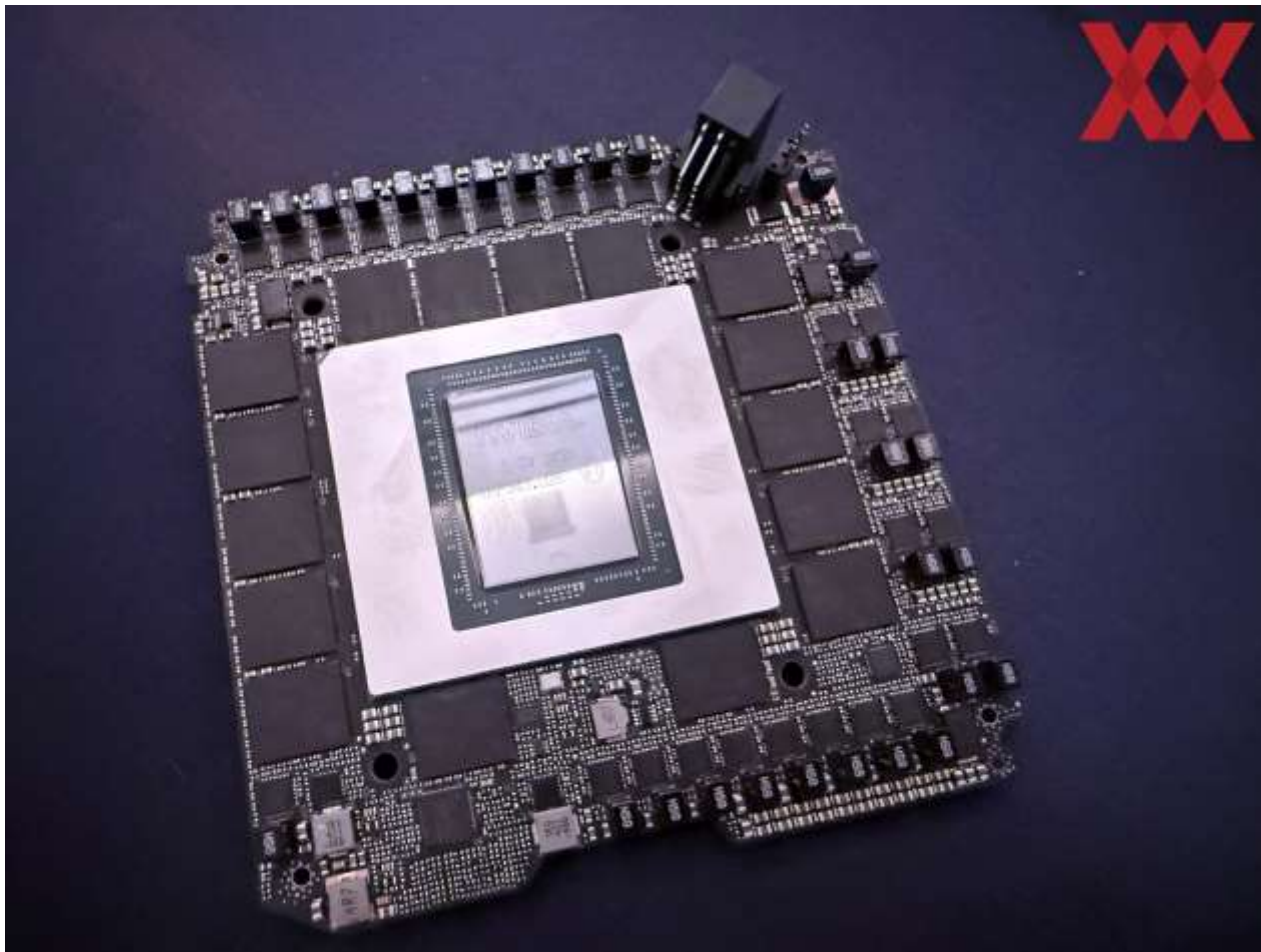
Эффективный инференс моделей





Как устроены GPU

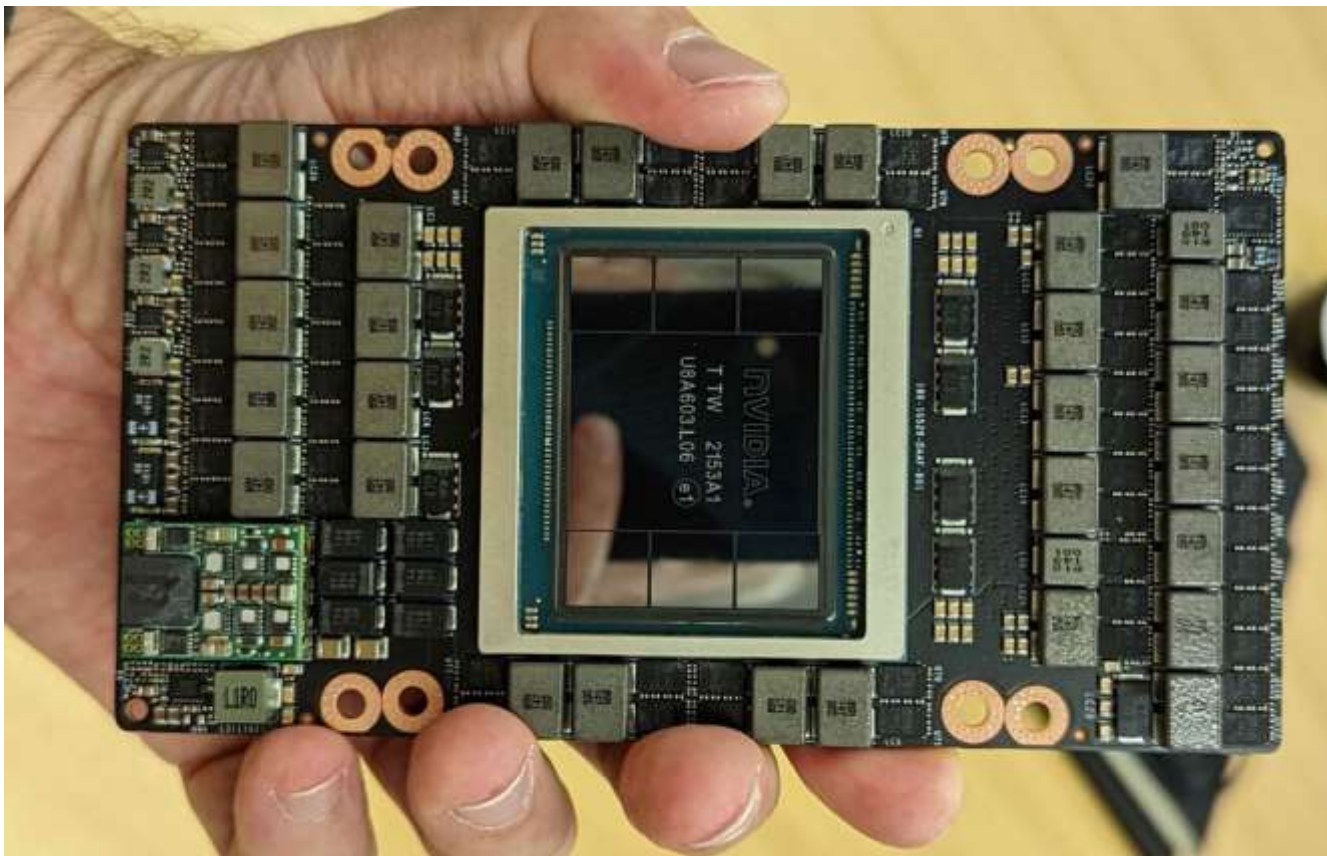
NVIDIA RTX 5090



- Чип GPU
- Память
- Шина

<https://www.hardwareluxx.de>

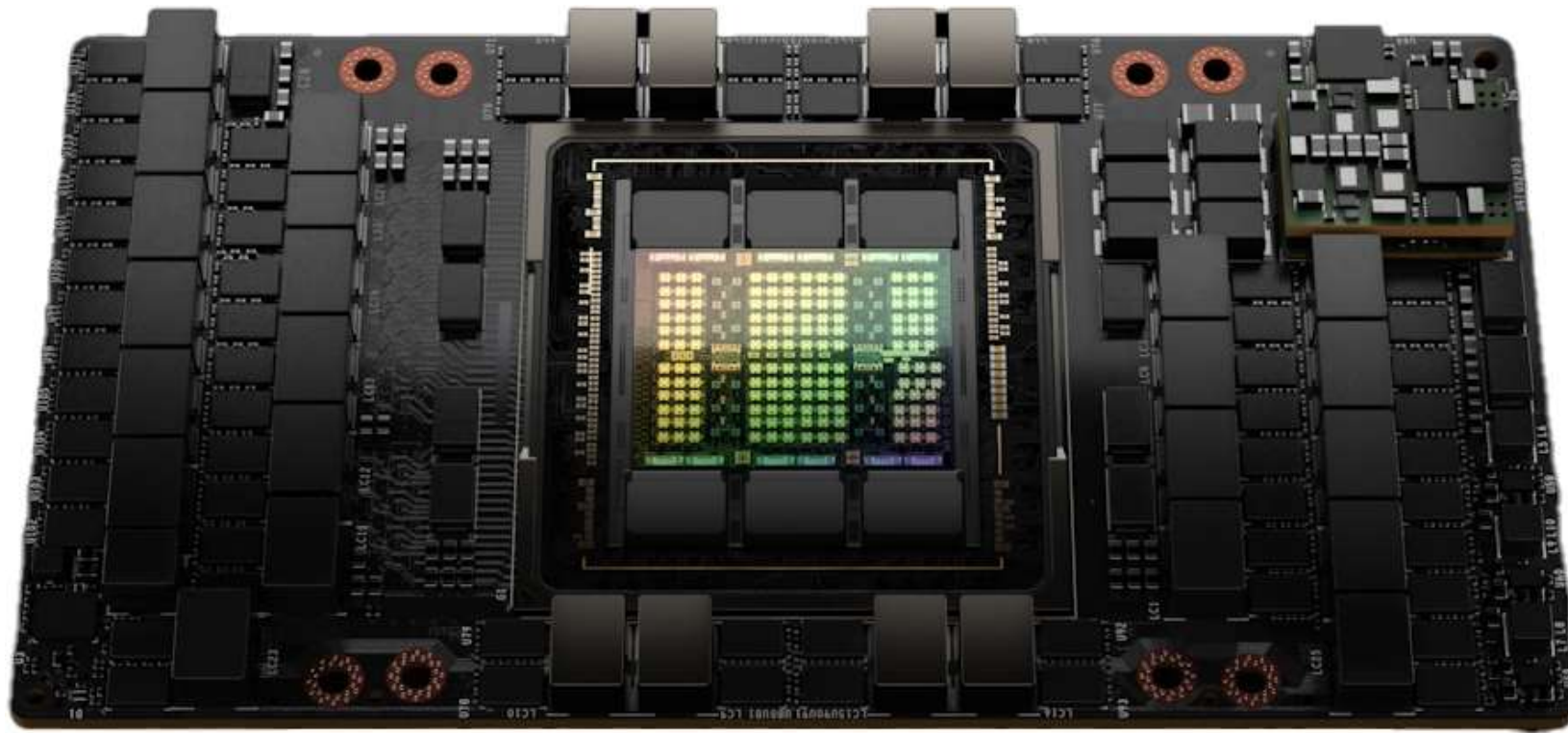
NVIDIA H100



- Чип GPU
- Память (HBM)
- Шина

<https://www.techpowerup.com/294588/nvidia-h100-sxm-hopper-gpu-pictured-up-close>

NVIDIA H100



<https://resources.nvidia.com/en-us-hpc-ai/nvidia-hopper-architecture>

NVIDIA H100, память и шина

- Память находится на чипе, но отделена от ядра
- ~2.5 TB/s пропускная способность шины
- 40 TPS для google/gemma-4-31B (fp16)
 - Memory-bandwidth-bound
- Общий объем: 80 GB

NVIDIA H100, GPU

- GPU заточены для параллельных вычислений
- H100 состоит из 132 Streaming Multiprocessor (SM)
 - Это аналог ядра в CPU

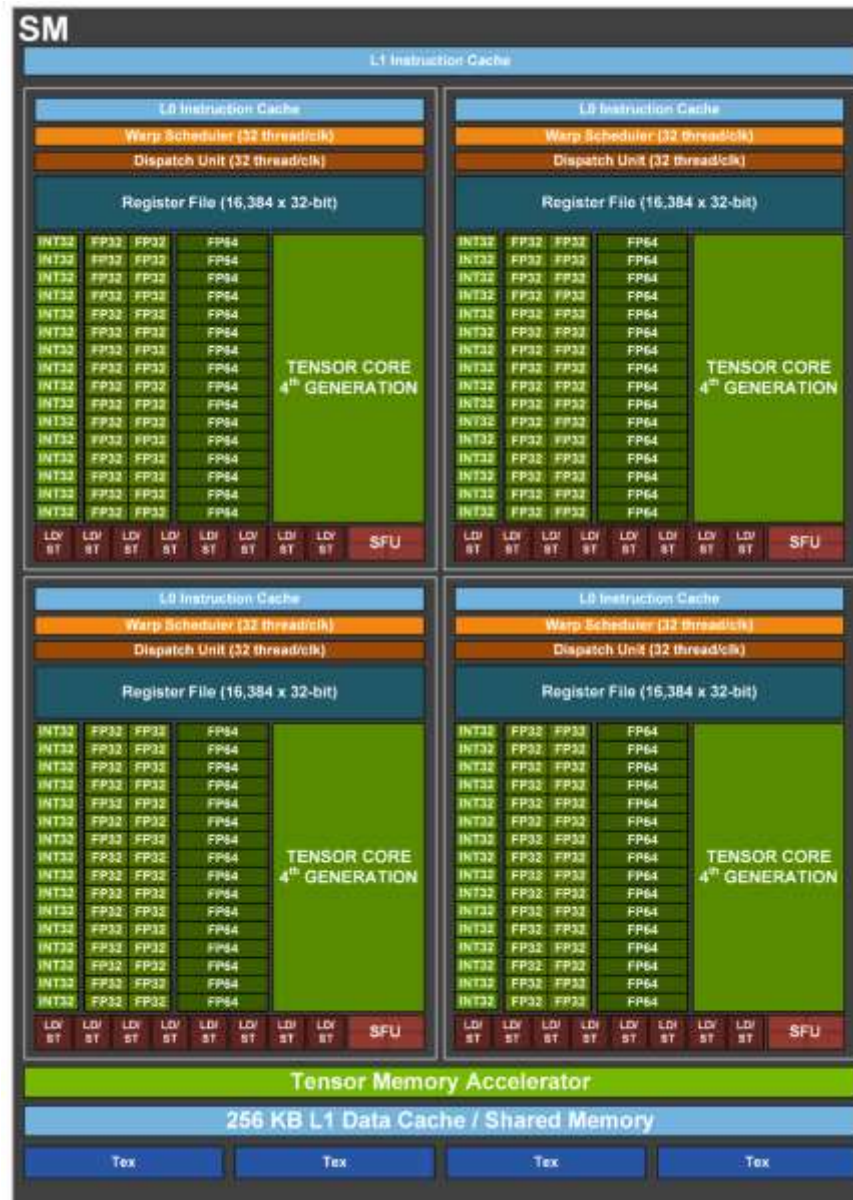
NVIDIA H100, GPU



NVIDIA H100, GPU

- В одном SM'e:
 - 128 CUDA ядер
 - 4 тензорных ядра
 - 4 шедулера

NVIDIA H100, GPU



- 4x Warp scheduler
- 4x Tensor core
- 128x CUDA core
 - 32 per scheduler

SIMT

- Концепция GPU: Single instruction, multiple threads
- Логика шедулинга инструкций дорогая
- В параллельных задачах делаем одно и то же много раз
- 1 шедулер на 32 исполнителя

SIMT

- Задача (грид) разбивается на произвольное число блоков
 - Блок попадает на SM
 - Внутри блока есть общая память
- Блок разбивается на 4 варпа (warp)
 - У каждого варпа свой шедулер
 - Варпы ждущие память могут вытесняться
- Варп разбивается на 32 треда
 - У каждого треда свои регистры, но общая инструкция
 - Тред выполняет минимальный квант работы

Пример CUDA кода

- BLOCK_SIZE константа для последующей аллокации

```
#define BLOCK_SIZE 256

void main() {
    // ...
    assert(BLOCK_SIZE == num_cols);
    int num_blocks = num_rows;
    rowMaxKernel<<num_blocks, BLOCK_SIZE>>>(d_mat, d_out);
}
```


Пример CUDA кода

```
__global__ void rowMaxKernel(const float* mat, float* out) {
    __shared__ float sdata[BLOCK_SIZE];

    int tid = threadIdx.x; // aka column, 0 : BLOCK_SIZE - 1
    int row = blockIdx.x; // 0 : num_blocks - 1

    sdata[tid] = mat[row * BLOCK_SIZE + tid];

    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = fmaxf(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }

    if (tid == 0) {
        out[row] = sdata[0];
    }
}
```

NVIDIA H100, GPU FLOPS

- Tensor:
 - 2000 TFLOPS fp8
 - 1000 TFLOPS fp16
- CUDA: 120 TFLOPS, fp16

Боттлнеки

- Объем VRAM
 - Сама модель
 - Кэши/батч-сайз
- Пропускная способность шины
 - Когда батч маленький
- Вычислительные возможности ядра
 - Когда батч большой

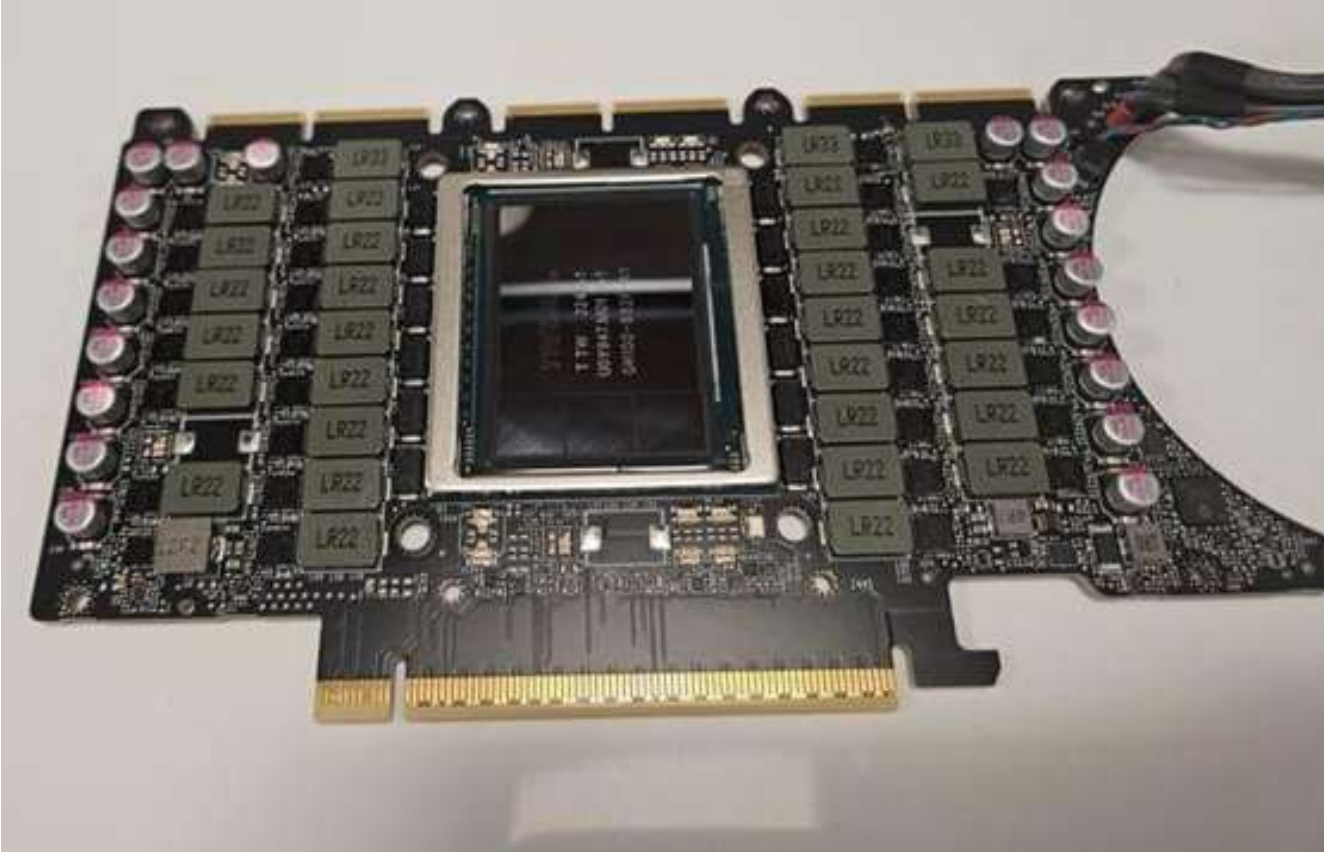
Пример: перемножение матриц

- Перемножение матрицы на вектор:
 - $O(N^2)$ память и вычисления (аналог $BS=1$)
- Перемножение двух матриц $N \times N$:
 - $O(N^2)$ память, $O(N^3)$ вычисления (аналог большого батча)
- FLOPS: $\sim 10^{15}$, Throughput $\sim 10^{12}$
 - При малом батче упираемся в Throughput
 - Большие батчи насыщают FLOPS за счет $O(N^3)$ вычислений
- На практике при переходе $BS=1 \rightarrow BS=2$ почти бесплатный



СВЯЗНОСТЬ

PCIe



- 64 GB/s
- Data-parallel
- Pipeline-parallel

u/Giga-Moose

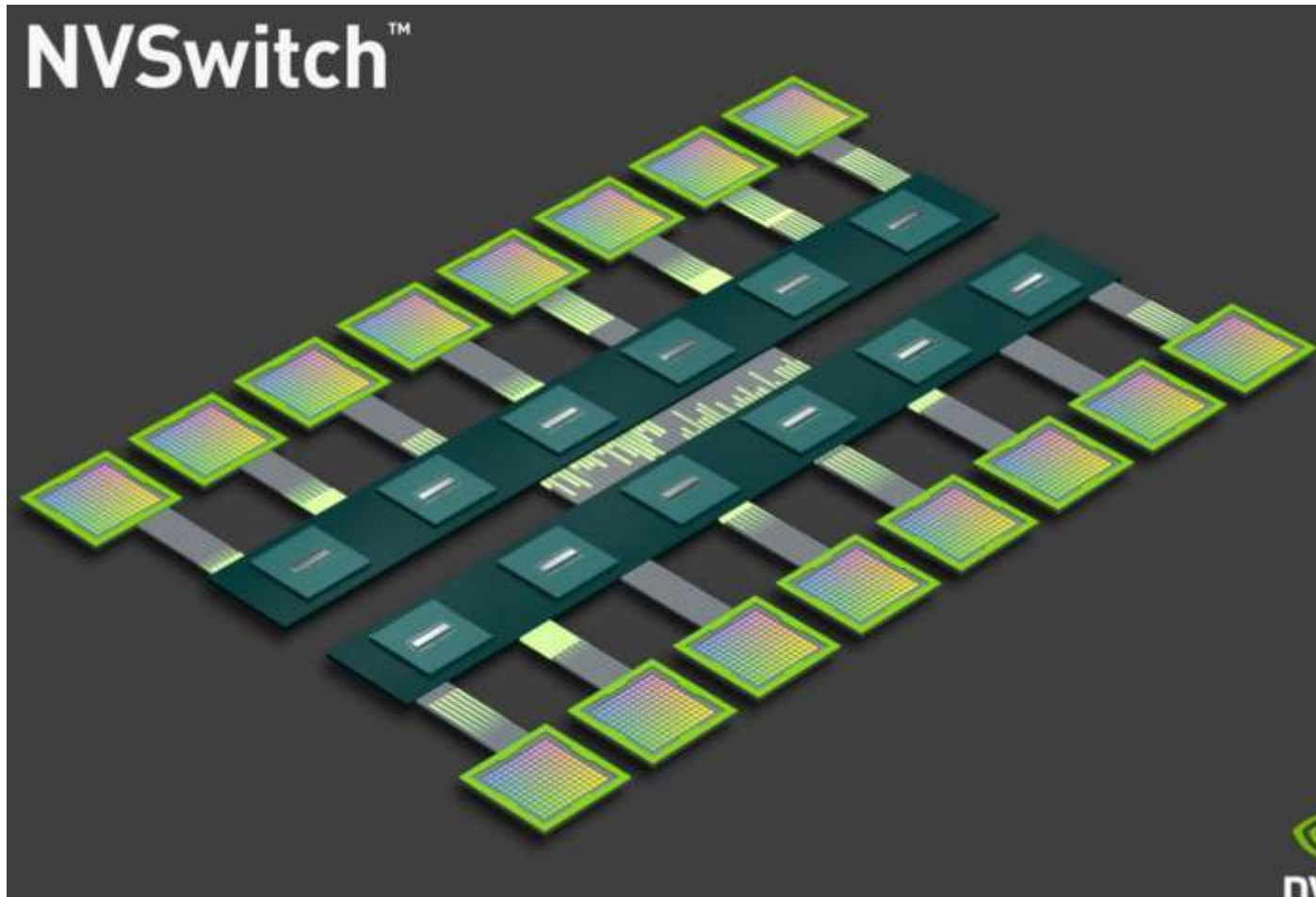
NVLink



- H100/H200: 900 GB/s
- B100/B200: 1800 GB/s
- Unified memory addressing
- Tensor-parallel
- Expert-parallel
- Point-to-Point

<https://serverflow.ru/blog/stati/nvlink-cto-eto-takoe-i-kak-rabotaet/>

NVSwitch



- All-to-all
- Внутри стойки/сервера

<https://www.scientific-computing.com/news/nvidia-unveils-dgx-2-and-nvswitch-gtc-conference>

InfiniBand

- Связь между серверами
- 25-50-100 GB/s на порт, ~8 портов
- Быстрый аналог Ethernet



Типы данных

FLOPS

TFLOPS	FP32	FP16	FP8	FP4	MAX
A100	150	300	-	-	300
H100	500	1000	2000	-	2000
B200	1100	2250	4500	9000	9000

Представление чисел с плавающей точкой

$$(-1)^S \cdot 2^{E-\max(E)/2} \cdot (1 + M)$$

Числа с плавающей точкой

	sign	exponent					mantissa										
FP16	0	0	1	1	0	1	1	0	0	1	0	1	0	0	1	1	= 0.395264
BF16	0	0	1	1	1	1	1	0	1	1	0	0	1	0	1	0	= 0.394531
FP8 E4M3	0	0	1	0	1	1	0	1									= 0.40625
FP8 E5M2	0	0	1	1	0	1	1	0									= 0.375

Требуют масштабирования!

Требуют
масштабирования!

https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html

mxfp4 / nvfp4

- e2m1: {0,±0.5,±1,±1.5,±2,±3,±4,±6}

$$x_{real} \approx S_b \cdot x_i^{(fp4)}$$

- mxfp4

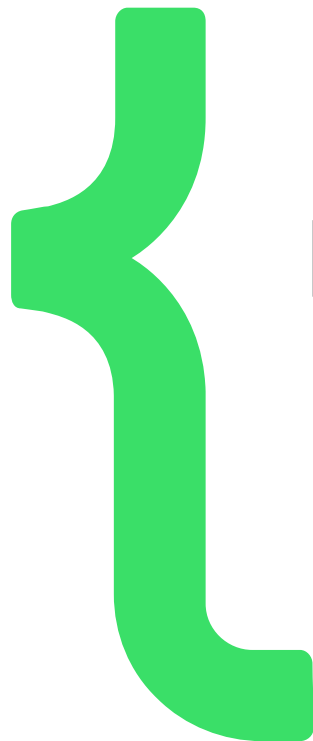
- 32 числа в блоке
- e8m0 масштаб блока
- gpt-oss

- nvfp4

- 16 в блоке
- e4m3 масштаб блока
- fp32 масштаб тензора
- говорят что чуть точнее mxfp4

Модели в mxfp4 / nvfp4

- mxfp4: openai/gpt-oss-120b
- nvfp4: nvidia/*-NVFP4 (Deepseek, GLM, Gemma, Qwen, ...)



Квантизации

Квантизация моделей

- Post training quantization
- Уменьшаем размер модели
 - Нужно меньше VRAM
 - Меньше нагрузка на шину
 - Больше места для кэша/батча

Линейная симметричная квантизация

$$b = 8, q_{min} = -127, q_{max} = 127$$

$$s = \frac{\max(|x|)}{2^{b-1} - 1} = \frac{\max(|x|)}{127}$$

$$x^{(Q)} = clip\left(\left[\frac{x^{(R)}}{s}\right]\right)$$

$$x^{(R)} \approx s \cdot x^{(Q)}$$

Линейная асимметричная квантизация

$$b = 8, \quad q_{min} = 0, \quad q_{max} = 255$$

$$s = \frac{\max(x) - \min(x)}{2^b - 1} = \frac{\max(x) - \min(x)}{255}$$

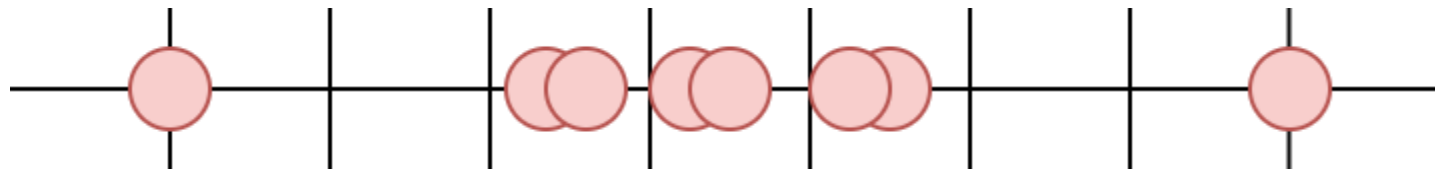
$$z = \left\lfloor \frac{\min(x)}{s} \right\rfloor$$

$$x^{(Q)} = \text{clip} \left(\left\lfloor \frac{x^{(R)}}{s} \right\rfloor - z \right)$$

$$x^{(R)} \approx s \cdot (x^{(Q)} + z)$$

Проблемы линейной квантизации

- Неэффективное заполнение пространства
- Требует дополнительных вычислений



FP8 квантизация

- Если сконвертировать bf16 в fp8 то потеряем точность
 - Проблема в диапазоне
- Будем хранить масштабировать каждый выходной канал!
- Еще хочется задействовать тензорные ядра
 - Надо умножать матрицы $\text{fp8} \times \text{fp8} \rightarrow \text{fp16}$

FP8 квантизация

$$W \in \mathbb{R}^{C_{out} \times C_{in}}$$

Свой для каждого
выходного канала!

$$s_i = \frac{\max_j |W_{i,j}|}{f_{max}}, f_{max} = 448$$

$$\hat{W}_{i,j} = \text{round}_{\text{FP8}} \left(\frac{W_{i,j}}{s_i} \right)$$

$$W_{i,j} \approx s_i \cdot \hat{W}_{i,j}$$

FP8 квантизация

$$X \in \mathbb{R}^{L \times C}$$

Свой для каждого
токена!

$$s_t = \frac{\max_c |X_{t,c}|}{f_{max}}$$

$$\hat{X}_{t,c} = \text{round}_{\text{FP8}} \left(\frac{X_{t,c}}{s_t} \right)$$

$$X_{t,c} \approx s_t \cdot \hat{X}_{t,c}$$

FP8 квантизация

$$\begin{aligned} A_{t,i} &\approx \sum_c s_t \cdot \hat{X}_{t,c} \cdot s_i \cdot \hat{W}_{i,c} = \\ &= s_t \cdot s_i \cdot \sum_c \hat{X}_{t,c} \cdot \hat{W}_{i,c} \end{aligned}$$

FP8 квантизация

- Поддерживается начиная с NVIDIA H100 (Hopper)
- Сжатие x4 по сравнению с fp32
- Минимальный оверхед, тк вынесли скейл наружу

AWQ квантизация

- У векторов x есть большие и малые компоненты
- Надо более точно квантовать входные каналы для больших компонент
- Можем переливать точность квантования масштабируя фичи

AWQ квантизация

$$\begin{aligned} A_{t,i} &= \sum_c X_{t,c} \cdot W_{i,c} = \\ &= \sum_c \hat{X}_{t,c} \cdot \hat{W}_{i,c} \end{aligned}$$

$$\hat{W}_{t,c} = s_t \cdot W_{t,c}$$

$$\hat{X}_{t,c} = X_{t,c} / s_t$$

AWQ квантизация

$$\bar{a}_i = \frac{1}{T} \sum_{t=1}^T |X_{t,i}|$$

$$s_i = \bar{a}_i^\alpha, \quad \alpha \in [0, 1]$$

$$\alpha^* = \arg \min_{\alpha} \|XW^\top - \hat{X}\hat{W}^\top\|$$

AWQ квантизация

- Слой который дает X называется `smooth_layer`
 - В него мы прячем деление на s , чтобы сэкономить вычисления
- Слой где считаем умножение на W называется `balance_layer`
 - Этот слой мы квантуем

AWQ квантизация

- Под капотом int4, поэтому требует доп. вычислений
- Оптимально для memory-bound
- В случае compute-bound лучше fp8/nvfp4

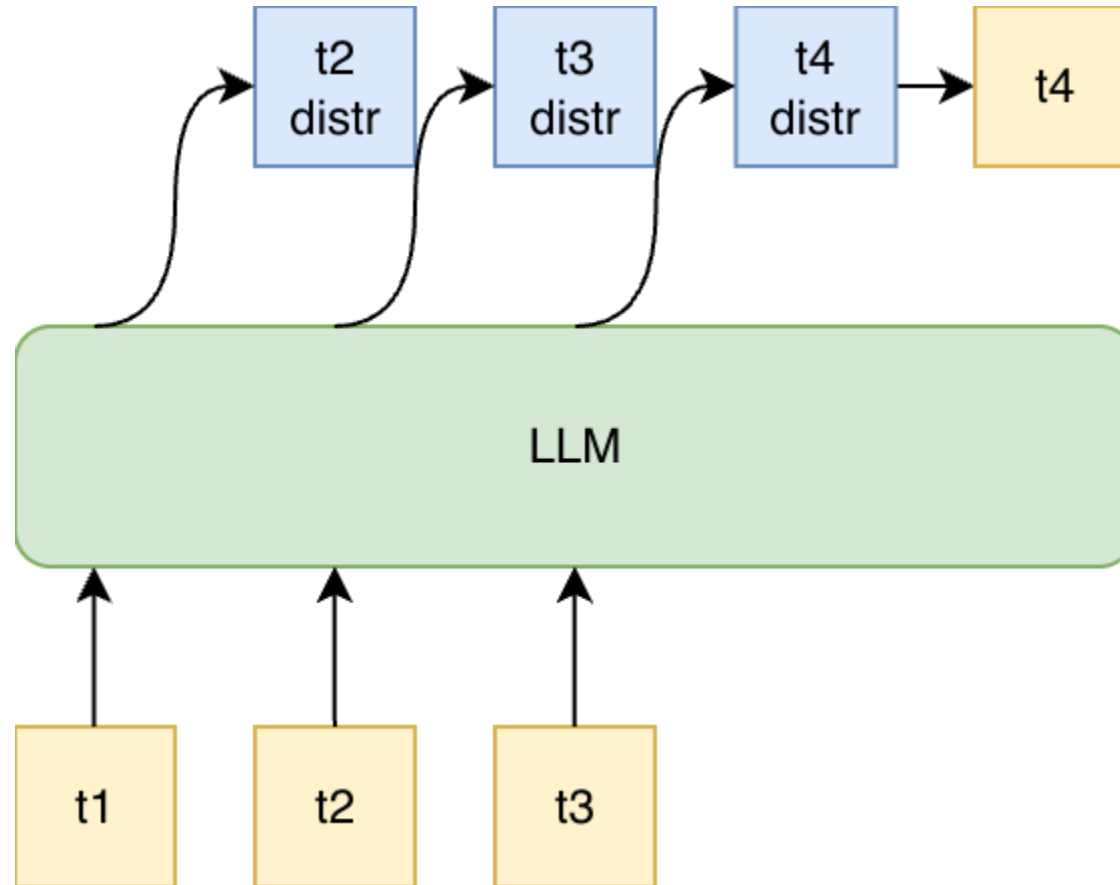
Примеры квантованных моделей

- fp16: google/gemma-4-31B
- fp8: deepseek-ai/DeepSeek-V3
- mxfp4: openai/gpt-oss-120b



Speculative Decoding

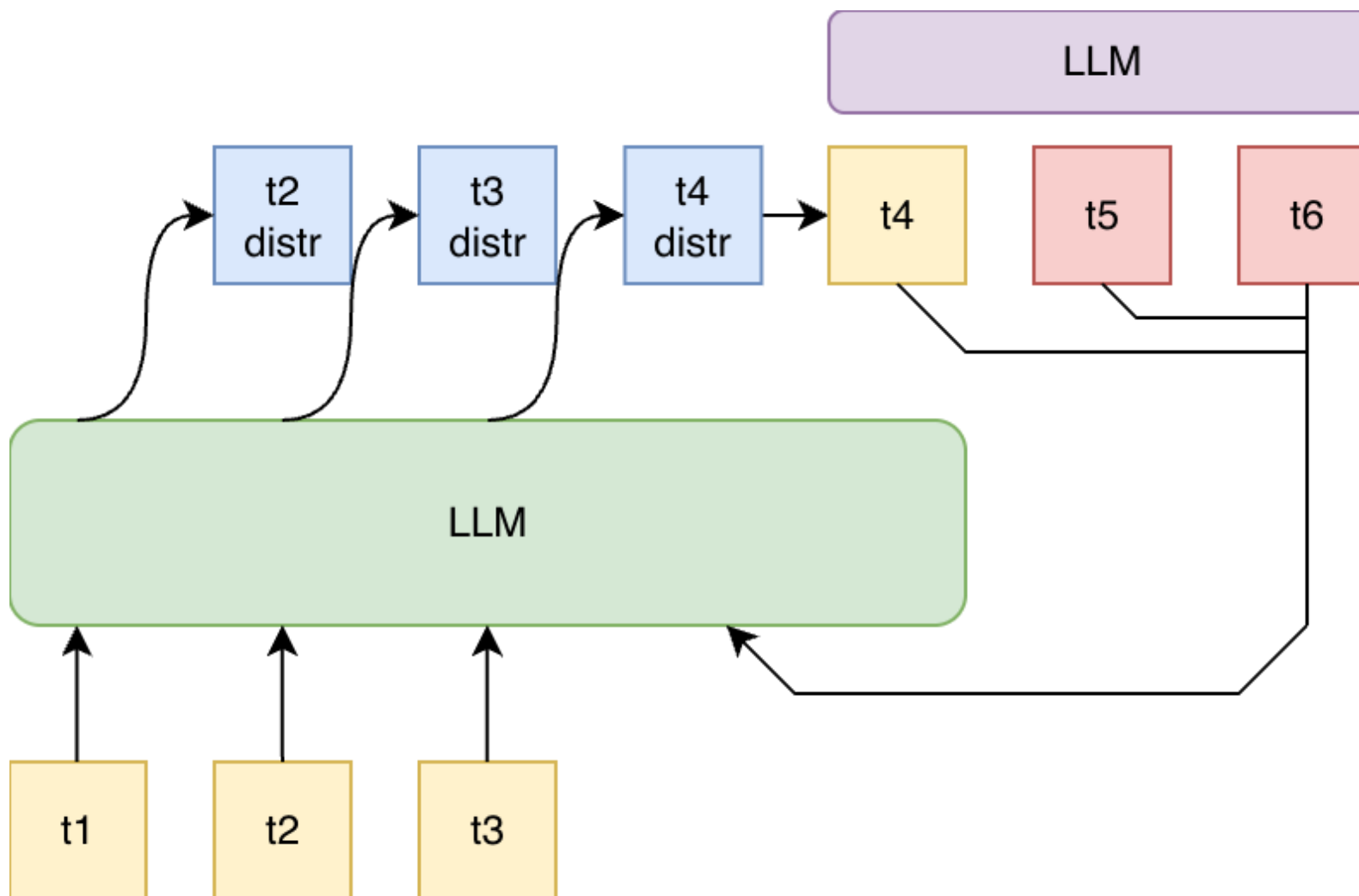
Как LLM генерирует токены



Speculative decoding

- Мы фидим токены по 1, тк имеем распределение только для 1
- Зафидить 1-2-4 занимает примерно то же самое время
 - Multi token prediction (MTP)
- Часто следующие токены очевидны
 - Можем предсказать быстрой моделью и проверить большой
 - Тривиально для argmax

Speculative decoding



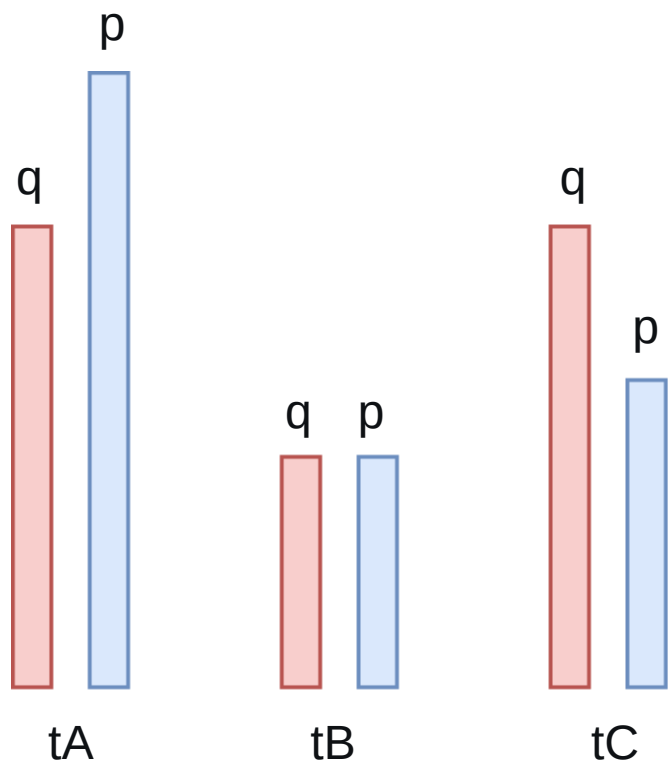
Speculative decoding

- Первый токен заведомо правильный
- Остальные валидируем
- Отбрасываем неправильные
- Если повезло, то за один вызов модели получили несколько токенов
- Но как быть с вероятностным сэмплением?

Speculative sampling

- Рассмотрим сценарий, когда сэмплируем, а не argmax
- Хотим сэмплировать из спекулятивной
- Хотим распределение как из большой
- Надо корректировать:
 - Отбрасываем некоторые токены
 - Пересэмплируем
- Хотим минимизировать количество отброшенных токенов

Speculative sampling



■ Вероятность из спекулятивной модели
■ Вероятность из большой модели

$$P_{accept} = \min \left(1, \frac{p(x_i)}{q(x_i)} \right)$$
$$p'(x) \propto \max(0, p(x) - q(x))$$

- Принимаем заниженные
- Переливаем из завышенных

Speculative sampling

- Чем ближе распределения, тем выше acceptance
- Чем выше acceptance, тем больше ускорение
- Обучим модель специально под целевую

EAGLE

- Однослойный трансформер
- Берем с целевой модели:
 - Первый слой
 - Слой из середины
 - Предпоследний слой
- Добавляем эмбединг
- Предсказываем распределение целевой модели

EAGLE: training-time test

- Хотим чтобы спек. модель могла в авторегрессию
- Обучаемся на 3-5 предсказанных последовательно токенах

EAGLE

Step	Input	Draft produces
1	F_t (from target) + $e(\text{tok}_t)$	$h_1, p_1 \rightarrow \text{tok}_{t+1}$
2	$h_1 + e(\text{tok}_{t+1})$	$h_2, p_2 \rightarrow \text{tok}_{t+2}$
3	$h_2 + e(\text{tok}_{t+2})$	$h_3, p_3 \rightarrow \text{tok}_{t+3}$

- На каждом шаге спек. модель видит результаты всех предыдущих шагов

Модели со встроенным MTP

- Deepseek V3.2: 80 TPS -> 140 TPS
- GLM-5: 70 TPS -> 100 TPS

Обучение EAGLE

- SpecForge или Speculators
- Нужно ~1 ГБ данных для обучения
 - Лучше из целевой модели
- Обучили для openai/gpt-oss-120b:
 - 210 TPS -> 270-300 TPS
- Основной прирост при малых BS



Адаптивные методы

Адаптивные методы

- Не все токены сложно предсказывать
- Будем применять всю модель только для сложных
- Для простых токенов модель должна сама отключать части себя
- Например слои или MoE эксперты

Early exit

- Считаем слои один за другим
- На каждом слое проверяем не можем ли уже предсказать токен
- Если можем, то предсказываем
 - Заполняем KV кэши высших слоев значениями текущего

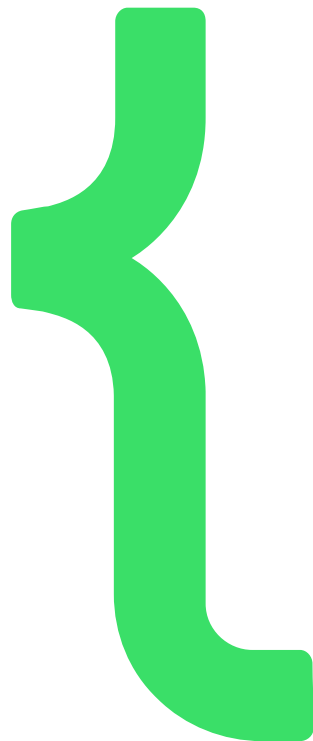
Early exit

$$H(p_i) = - \sum p_i \log(p_i) < \tau$$

$$P(top1) > \tau$$

$$P(top1) - P(top2) > \tau$$

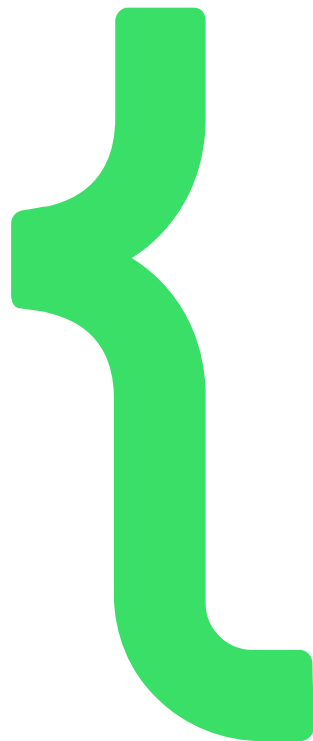
- Или просто обучаем доп. бинарный классификатор



Балансировка запросов

KV-cache aware routing

- У каждого пода свой кэш
- Хотим отправлять туда где наибольший хит
- Для этого есть глобальный балансер, который
 - Трэкает кэши воркеров
 - Токенизирует запрос
 - Отправляет на свободный воркер с наибольшим хитом префикса
- Принципиально важно для агентов/чатов



Метрики инференса

Метрики профиля запросов

- Можем трэкать
 - TPS total
 - TPS per user
 - RPS
- Но это метрики потока запросов
- Они не говорят об эффективности системы

Метрики vLLM

- Можно посмотреть на `/metrics`
- `request_prompt_tokens_bucket`
- `request_generation_tokens_bucket`
- `time_to_first_token_seconds_bucket`
- `e2e_request_latency_seconds_bucket`

Метрики vLLM

- num_requests_running
- num_requests_waiting
- kv_cache_usage_perc
- prefix_cache_hits_total / prefix_cache_queries_total



Метрики GPU

nvidia-smi

NVIDIA-SMI 570.211.01				Driver Version: 570.211.01				CUDA Version: 12.8			
GPU	Name			Persistence-M	Bus-Id	Disp.A	Volatile		Uncorr.	ECC	
Fan	Temp	Perf			Memory-Usage		GPU-Util	Compute	M.	M.	
				Pwr:Usage/Cap							
0	NVIDIA H100	80GB HBM3			00000000:19:00.0	Off			0		
N/A	56C	P0	700W / 700W		80295MiB / 81559MiB		100%		Default Disabled		

NVIDIA-SMI 570.211.01				Driver Version: 570.211.01				CUDA Version: 12.8			
GPU	Name			Persistence-M	Bus-Id	Disp.A	Volatile		Uncorr.	ECC	
Fan	Temp	Perf			Memory-Usage		GPU-Util	Compute	M.	M.	
				Pwr:Usage/Cap							
0	NVIDIA H100	80GB HBM3			00000000:19:00.0	Off			0		
N/A	49C	P0	432W / 700W		80295MiB / 81559MiB		100%		Default Disabled		

dcgm-exporter

- docker run ... nvcr.io/nvidia/k8s/dcgm-exporter:latest -f dcp-metrics.csv

dcp-metrics.csv:

...

DCGM_FI_PROF_PIPE_FP16_ACTIVE, gauge, Ratio of cycles the fp16 pipes are active (in %).

...

dcgm-exporter

DCGM_FI_PROF_GR_ENGINE_ACTIVE

DCGM_FI_DEV_POWER_USAGE

DCGM_FI_PROF_SM_ACTIVE

dcgm-exporter

DCGM_FI_PROF_PIPE_TENSOR_ACTIVE

DCGM_FI_PROF_PIPE_FP16_ACTIVE

DCGM_FI_PROF_PIPE_FP32_ACTIVE

DCGM_FI_PROF_DRAM_ACTIVE

dcgm-exporter

DCGM_FI_DEV_CLOCK_THROTTLE_REASONS



Спасибо за внимание!

